

Resolução da Prova de Inteligência Artificial

Ana Lilian Alfonso Toledo¹

¹Curso de Ciência da Computação - Universidade Federal de Santa Maria (UFSM)

altoledo@inf.ufsm.br

Convenções usadas na resolução: quando falo em “nó expandido/selecionado”, considero o nó retirado da fronteira para teste. O nó objetivo aparece na ordem de seleção, mas não gera sucessores. Em “nós gerados”, conto também o nó inicial. Nos empates, usei a ordem de geração indicada no enunciado; em A*, usei menor f , depois menor h , depois ordem de geração.

1. Questão 01 – Busca em amplitude/largura, Busca em profundidade e Busca iterativa em profundidade

1.1. Modelagem

- a) **Definição formal do estado:** os estados constituem as salas em que o agente pode estar. Essas salas estão representadas na tabela e vão de A até S.
- b) **Estado inicial:** A.
- c) **Teste de objetivo:** Estado atual == S.
- d) **Função sucessora:** retorna as salas vizinhas de cada sala respeitando a ordem obrigatória da tabela.
- e) **Função de custo:** todas as ações possuem custo unitário; logo, o custo da solução é o número de arestas do caminho.
- f) **Representação em Python:** dicionário sala -> lista de sucessores.

Grafo utilizado:

A: B, C, D
B: E, F
C: G, H
D: I
E: J
F: K, L
G: M
H: N, O
I: P
J: -
K: Q
L: -
M: R
N: -
O: S
P: -
Q: -
R: -
S: -

1.2. Busca em amplitude/largura — BFS

A BFS expande por níveis. Como todos os custos são unitários, encontra a solução de menor profundidade.

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[C, D, E, F]
3	C	[D, E, F, G, H]
4	D	[E, F, G, H, I]
5	E	[F, G, H, I, J]
6	F	[G, H, I, J, K, L]
7	G	[H, I, J, K, L, M]
8	H	[I, J, K, L, M, N, O]
9	I	[J, K, L, M, N, O, P]
10	J	[K, L, M, N, O, P]
11	K	[L, M, N, O, P, Q]
12	L	[M, N, O, P, Q]
13	M	[N, O, P, Q, R]
14	N	[O, P, Q, R]
15	O	[P, Q, R, S]
16	P	[Q, R, S]
17	Q	[R, S]
18	R	[S]
19	S	[]

c) **Árvore parcial de busca:**

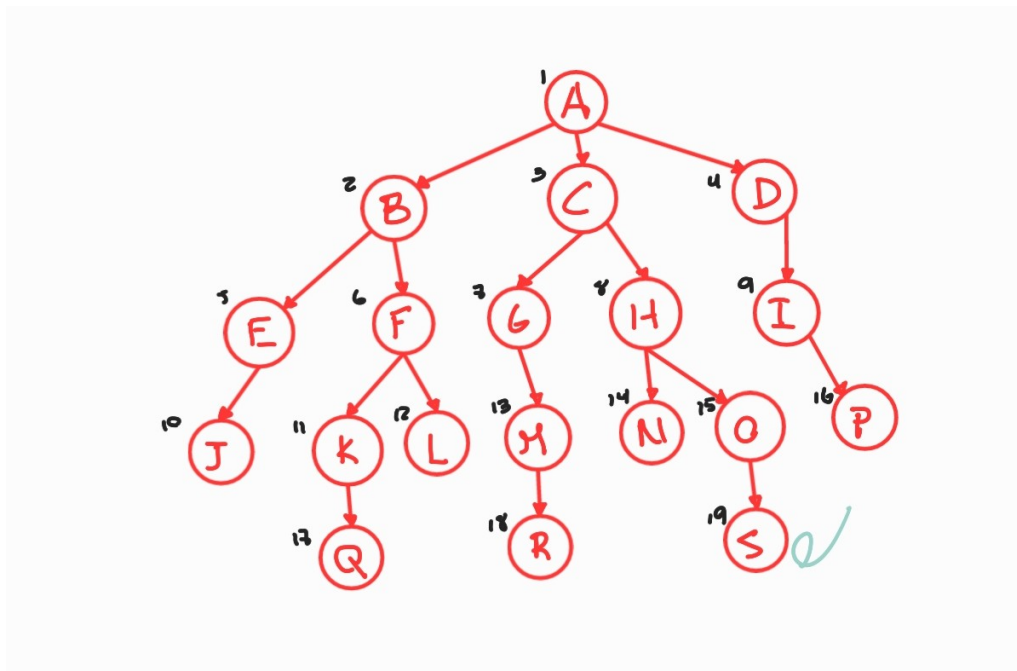


Figura 1. Árvore de busca em amplitude/largura

d) **Caminho solução:** A → C → H → O → S.

e) **Profundidade:** 4.

- f) **Custo:** 4.
- g) **Nós gerados:** 19.
- h) **Nós selecionados/expandidos:** 19.

1.3. Busca em profundidade — DFS

A DFS segue o primeiro sucessor até o fim antes de retroceder.

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[Q, L, C, D]
7	Q	[L, C, D]
8	L	[C, D]
9	C	[G, H, D]
10	G	[M, H, D]
11	M	[R, H, D]
12	R	[H, D]
13	H	[N, O, D]
14	N	[O, D]
15	O	[S, D]
16	S	[D]

c) **Árvore parcial de busca:**

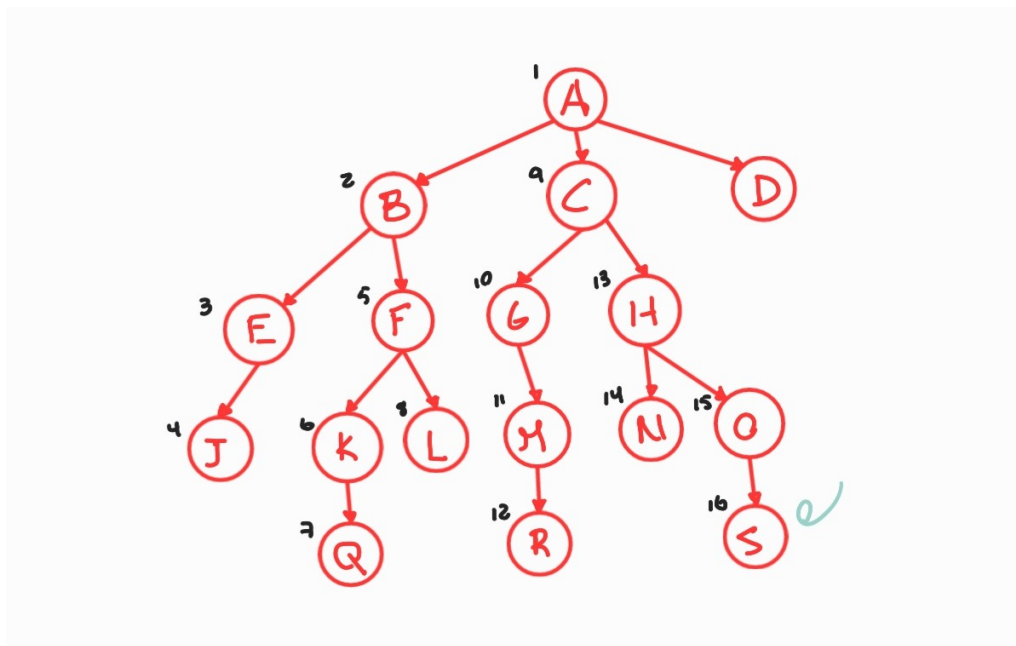


Figura 2. Árvore de busca em profundidade

- d) **Caminho solução:** A -> C -> H -> O -> S.
- e) **Profundidade:** 4.

- f) **Custo:** 4.
- g) **Nós gerados:** 17.
- h) **Nós selecionados/expandidos:** 16.

1.4. Busca iterativa em profundidade — IDS

A IDS executa buscas limitadas em profundidade com limites crescentes.

Limite 0

Passo	Nó selecionado	Fronteira após expansão
1	A	[]

Limite 1

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[C, D]
3	C	[D]
4	D	[]

Limite 2

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[F, C, D]
4	F	[C, D]
5	C	[G, H, D]
6	G	[H, D]
7	H	[D]
8	D	[I]
9	I	[]

Limite 3

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[L, C, D]
7	L	[C, D]
8	C	[G, H, D]
9	G	[M, H, D]
10	M	[H, D]
11	H	[N, O, D]
12	N	[O, D]

Limite 3

Passo	Nó selecionado	Fronteira após expansão
13	O	[D]
14	D	[I]
15	I	[P]
16	P	[]

Limite 4

Passo	Nó selecionado	Fronteira após expansão
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[Q, L, C, D]
7	Q	[L, C, D]
8	L	[C, D]
9	C	[G, H, D]
10	G	[M, H, D]
11	M	[R, H, D]
12	R	[H, D]
13	H	[N, O, D]
14	N	[O, D]
15	O	[S, D]
16	S	[D]

c) **Árvore parcial de busca:**

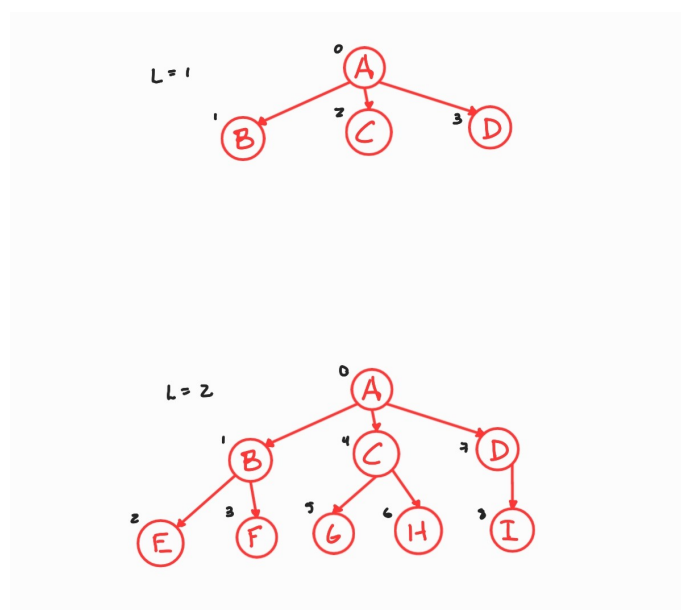


Figura 3. Árvore busca iterativa em profundidade (limites 1 e 2)

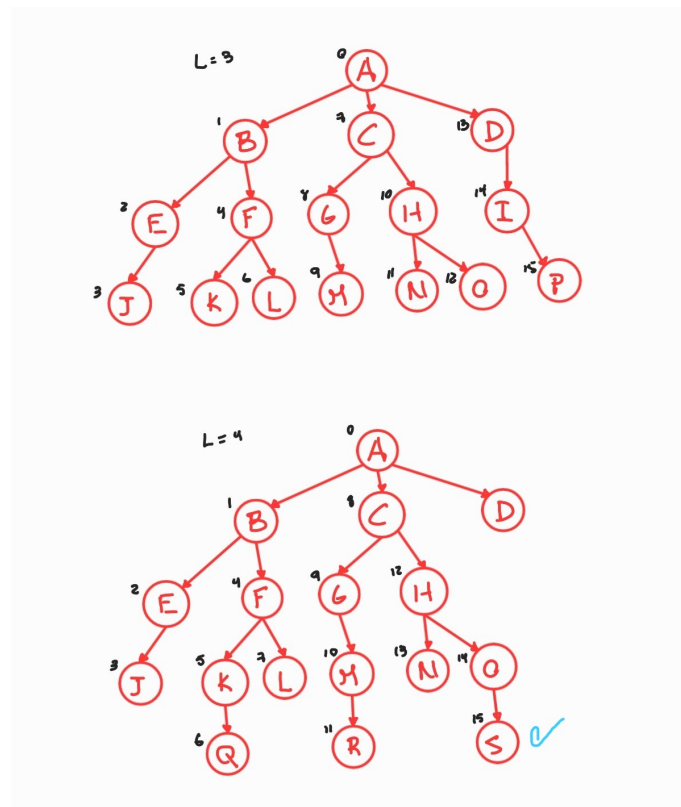


Figura 4. Árvore busca iterativa em profundidade (limites 3 e 4)

- d) **Caminho solução:** A $\rightarrow$ C $\rightarrow$ H $\rightarrow$ O $\rightarrow$ S.
- e) **Profundidade:** 4.
- f) **Custo:** 4.
- g) **Total de nós gerados nas iterações:** 47.
- h) **Total de nós selecionados/expandidos nas iterações:** 46.

1.5. Experimento: alteração da ordem dos sucessores

Foram alteradas exatamente duas listas de sucessores:

C: G, H $\rightarrow$ H, G
H: N, O $\rightarrow$ O, N

Resultados:

Algoritmo	Caminho solução	Nós gerados	Nós selecionados	Impacto
BFS	A-C-H-O-S	19	18	Pequena melhora, pois ainda explora por nível.
DFS	A-C-H-O-S	15	12	Melhora clara, pois passa a entrar primeiro no ramo que leva a S.

Algoritmo	Caminho solução	Nós gerados	Nós selecionados	Impacto
IDS	A-C-H-O-S	45	42	Melhora moderada, principalmente na última iteração.

- a) **O caminho solução mudou?** Não. O caminho solução continuou sendo: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$.
- b) **A quantidade de nós expandidos mudou?** Sim, diminuiu. Considerando o nó objetivo como selecionado/expandido, obteve-se: BFS: $19 \rightarrow 18$, DFS: $16 \rightarrow 12$, IDS: $46 \rightarrow 42$.
- c) **Algum algoritmo foi mais sensível à ordem dos sucessores?** Sim. O algoritmo mais sensível foi o DFS, pois ele aprofunda sempre pelo primeiro sucessor disponível. O IDS também foi afetado, pois utiliza buscas em profundidade limitadas. Já o BFS foi menos sensível à alteração.
- d) **Qual algoritmo apresentou menor consumo de memória?** O DFS apresentou o menor consumo de memória, pois mantém principalmente o caminho atual e alguns nós pendentes. O IDS também possui baixo consumo de memória, enquanto o BFS consome mais por armazenar muitos nós na fronteira.
- e) **Qual algoritmo encontrou a solução mais rapidamente?** Com a nova ordem dos sucessores, o DFS encontrou a solução mais rapidamente, pois alcançou o ramo correto com menos expansões.

1.6. Comparação dos três algoritmos

Critério	BFS	DFS	IDS
Compleitude	Sim	Sim, em grafo finito sem ciclos	Sim
Otimidade	Sim, com custo unitário	Não, em geral	Sim, com custo unitário
Complexidade de tempo	$O(b^d)$	$O(b^m)$	$O(b^d)$
Complexidade de espaço	$O(b^d)$	$O(b \cdot m)$	$O(b \cdot d)$
Dependência da ordem dos sucessores	Baixa	Alta	Moderada
Comportamento em árvores profundas	Ruim, devido ao consumo de memória	Pode não terminar se a árvore tiver profundidade infinita	Bom
Adequação para problemas grandes	Ruim	Moderada	Boa

2. Questão 02 — Busca Gulosa pela melhor Escolha e A*

2.1. Modelagem

- c) **Representação dos estados:** Os estados constituem as posições em que o agente pode se encontrar. Essas posições estão representadas na tabela e vão de A até T.

- d) **Estado inicial:** A.
- e) **Teste de objetivo:** Estado atual == T..
- f) **Função sucessora:** retorna as posições sucessoras de cada posição e seus respectivos custos respeitando a ordem da tabela
- g) **Função de custo:** $g(n)$, soma dos custos das arestas percorridas desde o estado inicial até n
- h) **Heurística:** $h(n)$, estimativa da distância restante até T.

2.2. Busca Gulosa pela Melhor Escolha

A busca gulosa utiliza:

$$f(n) = h(n)$$

Passo	Nó selecionado	$g(n)$	$h(n)$	$f(n)$	Fronteira ordenada por f
1	A	0	10	10	C(7), B(8), D(9)
2	C	4	7	7	H(4), G(6), B(8), D(9)
3	H	10	4	4	O(1), N(4), G(6), B(8), D(9)
4	O	14	1	1	T(0), N(4), G(6), B(8), D(9)
5	T	19	0	0	—

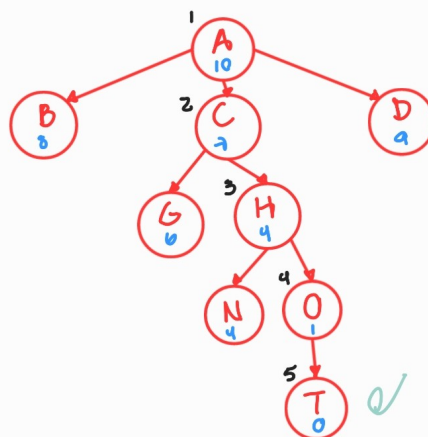


Figura 5. Árvore busca gulosa pela melhor escolha

- d) **Caminho encontrado:** A -> C -> H -> O -> T.
- e) **Custo final:** $4 + 6 + 4 + 5 = 19$.
- f) **Nós gerados:** 9.
- g) **Nós selecionados/expandidos:** 5.

2.3. Busca A*

A busca A* utiliza:

$$f(n) = g(n) + h(n)$$

Passo	Nó selecionado	g(n)	h(n)	f(n)	Fronteira ordenada por f
1	A	0	10	10	B(f=10), C(f=11), D(f=12)
2	B	2	8	10	E(11), C(11), F(12), D(12)
3	E	5	6	11	C(11), F(12), D(12), J(14)
4	C	4	7	11	F(12), D(12), H(14), J(14), G(14)
5	F	7	5	12	D(12), K(13), H(14), J(14), G(14), L(18)
6	D	3	9	12	I(12), K(13), H(14), J(14), G(14), L(18)
7	I	5	7	12	K(13), H(14), J(14), G(14), L(18), P(18)
8	K	10	3	13	H(14), J(14), G(14), R(15), L(18), P(18)
9	H	10	4	14	J(14), G(14), O(15), R(15), N(17), ...
10	J	9	5	14	G(14), O(15), R(15), N(17), Q(17), ...
11	G	8	6	14	O(15), R(15), M(17), N(17), Q(17), ...
12	O	14	1	15	R(15), M(17), N(17), Q(17), T(19), ...
13	R	13	2	15	T(17), M(17), N(17), Q(17), ...
14	T	17	0	17	—

d) **Caminho encontrado:** A → B → F → K → R → T.

e) **Custo final:** 2 + 5 + 3 + 3 + 4 = 17.

f) **Nós gerados:** 19 (o nó T é contado apenas uma vez).

g) **Nós selecionados/expandidos:** 14.

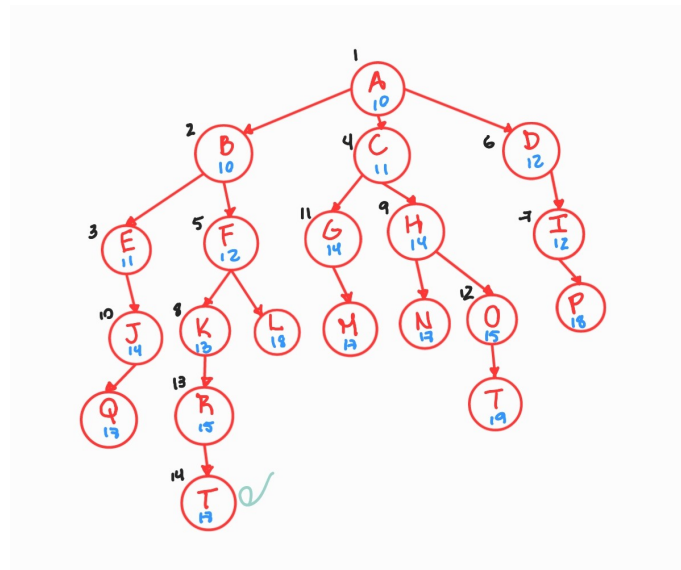


Figura 6. Árvore busca A*

2.4. Modificação de três valores heurísticos

Foram alterados exatamente três valores:

$h(B) : 8 \rightarrow 2$
 $h(C) : 7 \rightarrow 12$
 $h(R) : 2 \rightarrow 0$

Resultados:

Algoritmo	Caminho encontrado	Custo	Nós gerados	Nós selecionados/expandidos	Impacto
Gulosa	A-B-F-K-R-T	17	10	6	Mudou a decisão e passou a encontrar o caminho ótimo.
A*	A-B-F-K-R-T	17	19	14	A solução permaneceu ótima; mudou apenas a ordem de algumas expansões.

- A solução encontrada mudou?** Sim, para a Busca Gulosa. Antes, ela seguia: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow T$. Após a modificação da heurística, passou a seguir: $A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$. No caso do A*, a solução encontrada não mudou.
- O custo da solução mudou?** Sim, para a Busca Gulosa. O custo passou de: $19 \rightarrow 17$. Para o A*, o custo permaneceu: 17.
- A Busca Gulosa encontrou solução ótima?** Sim. Com a heurística modificada, a Busca Gulosa encontrou o caminho: $A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$, cujo custo é 17, que corresponde à solução ótima.

- d) **O A* encontrou solução ótima?** Sim. O A* encontrou a solução ótima: $A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$, com custo total 17.
- e) **Qual algoritmo expandiu menos nós?** A Busca Gulosa expandiu menos nós.
- f) **Qual algoritmo foi mais sensível à heurística?** A Busca Gulosa foi mais sensível, pois sua decisão depende apenas de $h(n)$. Já o A* usa: $f(n) = g(n) + h(n)$, combinando o custo acumulado com a heurística, o que torna sua decisão mais estável.

2.5. Comparação dos dois algoritmos

Critério	Greedy Best-First	A*
Completude	Não, pode entrar em ciclo sem checagem	Sim, com lista fechada
Otimidade	Não	Sim, com h admissível
Consumo de memória	$O(b^m)$	$O(b^d)$
Dependência da heurística	Total, pois usa apenas h	Parcial, pois usa $g + h$
Custo computacional	Baixo	Maior
Qualidade das soluções	Pode ser subótima	Ótima
Comportamento em grafos grandes	Rápido, mas impreciso	Lento, mas preciso

3. Questão 03 — Busca Local no problema das 8-rainhas

3.1. Modelagem

- a) **Representação do estado:** vetor $[s_1, s_2, \dots, s_8]$, em que s_i representa a linha da rainha na coluna i .
- b) **Definição de vizinho:** mover exatamente uma rainha para outra linha da mesma coluna.
- c) **Função de avaliação:** $h(s)$ = número total de pares de rainhas em conflito.
- d) **Critério de parada:** Parar quando $h(s) = 0$ (solução encontrada) ou quando nenhum vizinho tem h menor que o estado atual (máximo local)
- e) **Interpretação da superfície de busca:** Minimizar $h(s)$. O algoritmo desce pela superfície tentando encontrar $h(s) = 0$.

3.2. Hill-Climbing simples

Critério utilizado: escolher o vizinho com menor $h(s)$. Em empate, usar menor coluna e depois menor linha.

Iter.	Estado atual	$h(s)$	5 melhores vizinhos gerados	Melhor vizinho escolhido
0	[1, 1, 1, 1, 1, 1, 1, 1]	28	[1, 8, 1, 1, 1, 1, 1, 1], $h = 21$ [1, 1, 7, 1, 1, 1, 1, 1], $h = 21$ [1, 1, 8, 1, 1, 1, 1, 1], $h = 21$ [1, 1, 1, 6, 1, 1, 1, 1], $h = 21$ [1, 1, 1, 7, 1, 1, 1, 1], $h = 21$	[1, 8, 1, 1, 1, 1, 1, 1]
1	[1, 8, 1, 1, 1, 1, 1, 1]	21	[2, 8, 1, 1, 1, 1, 1, 1], $h = 15$ [1, 8, 1, 7, 1, 1, 1, 1], $h = 15$ [1, 8, 1, 1, 6, 1, 1, 1], $h = 15$ [1, 8, 1, 1, 7, 1, 1, 1], $h = 15$ [1, 8, 1, 1, 1, 5, 1, 1], $h = 15$	[2, 8, 1, 1, 1, 1, 1, 1]
2	[2, 8, 1, 1, 1, 1, 1, 1]	15	[2, 8, 1, 7, 1, 1, 1, 1], $h = 10$ [2, 8, 1, 1, 7, 1, 1, 1], $h = 10$ [2, 8, 1, 1, 1, 5, 1, 1], $h = 10$ [2, 8, 1, 1, 1, 6, 1, 1], $h = 10$ [2, 8, 1, 1, 1, 1, 6, 1], $h = 10$	[2, 8, 1, 7, 1, 1, 1, 1]
3	[2, 8, 1, 7, 1, 1, 1, 1]	10	[2, 8, 1, 7, 1, 6, 1, 1], $h = 6$ [2, 8, 1, 7, 1, 1, 6, 1], $h = 6$ [2, 8, 1, 7, 1, 1, 1, 5], $h = 6$ [2, 8, 2, 7, 1, 1, 1, 1], $h = 7$ [2, 8, 3, 7, 1, 1, 1, 1], $h = 7$	[2, 8, 1, 7, 1, 6, 1, 1]
4	[2, 8, 1, 7, 1, 6, 1, 1]	6	[2, 8, 1, 7, 1, 6, 1, 5], $h = 3$ [2, 8, 2, 7, 1, 6, 1, 1], $h = 4$ [2, 8, 4, 7, 1, 6, 1, 1], $h = 4$ [2, 8, 5, 7, 1, 6, 1, 1], $h = 4$ [2, 8, 1, 7, 2, 6, 1, 1], $h = 4$	[2, 8, 1, 7, 1, 6, 1, 5]
5	[2, 8, 1, 7, 1, 6, 1, 5]	3	[2, 8, 1, 7, 4, 6, 1, 5], $h = 1$ [2, 8, 2, 7, 1, 6, 1, 5], $h = 2$ [2, 8, 4, 7, 1, 6, 1, 5], $h = 2$ [2, 8, 1, 7, 1, 6, 2, 5], $h = 2$ [2, 8, 3, 7, 1, 6, 1, 5], $h = 3$	[2, 8, 1, 7, 4, 6, 1, 5]
6	[2, 8, 1, 7, 4, 6, 1, 5]	1	[2, 8, 3, 7, 4, 6, 1, 5], $h = 1$ [2, 8, 1, 7, 4, 6, 3, 5], $h = 1$ [3, 8, 1, 7, 4, 6, 1, 5], $h = 2$ [5, 8, 1, 7, 4, 6, 1, 5], $h = 2$ [6, 8, 1, 7, 4, 6, 1, 5], $h = 2$	Não há vizinho melhor; melhor vizinho também tem $h = 1$

O algoritmo para em [2, 8, 1, 7, 4, 6, 1, 5], com $h(s) = 1$ pois ocorreu um mínimo local.

- Por que ocorreu:** O mínimo local ocorreu pois nenhum vizinho melhora a avaliação. Como o objetivo é minimizar conflitos, o equivalente aos “máximos locais” do enunciado é um mínimo local.
- Como isso afetou a busca:** O algoritmo parou a busca sem achar o estado objetivo (não foi completo).
- Como poderia ser evitado:** Usando uma variação do Hill-Climbing, o Random Restar Hill-Climbing, ou usando Simulated Annealing

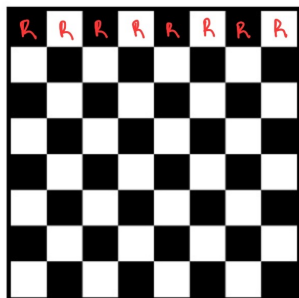


Figura 7. Árvore de busca Hill-Climbing simples 1

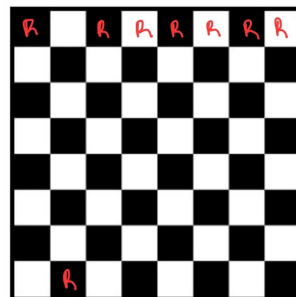


Figura 8. Árvore de busca Hill-Climbing simples 2

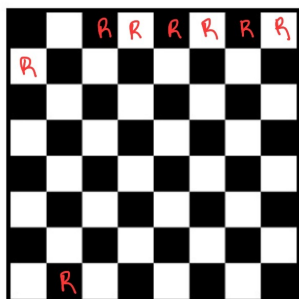


Figura 9. Árvore de busca Hill-Climbing simples 3

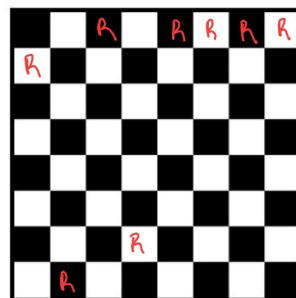


Figura 10. Árvore de busca Hill-Climbing simples 4

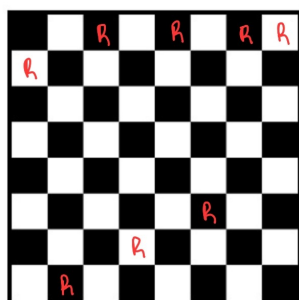


Figura 11. Árvore de busca Hill-Climbing simples 5

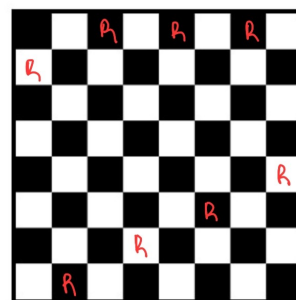


Figura 12. Árvore de busca Hill-Climbing simples 6

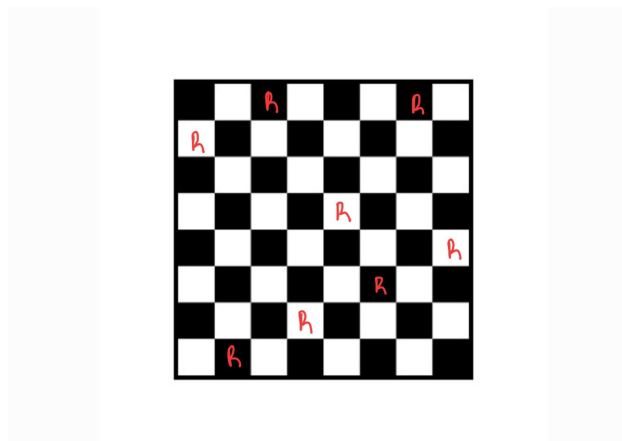


Figura 13. Árvore de busca Hill-Climbing simples 7

3.3. Random Restart Hill-Climbing — 20 execuções

Com semente fixa 42, sem movimentos laterais:

Exec.	Estado inicial	Passos	h(s) final	Solução?
1	[2,1,5,4,4,3,2,2]	4	1	Não
2	[7,1,1,2,4,4,1,4]	5	0	Sim
3	[7,4,8,5,1,3,7,6]	2	2	Não
4	[5,3,4,6,2,2,7,2]	2	1	Não
5	[6,6,5,1,8,2,7,2]	4	0	Sim
6	[5,6,4,2,1,4,5,2]	3	1	Não
7	[4,2,7,5,8,6,3,6]	1	2	Não
8	[6,4,5,2,3,4,3,8]	3	1	Não
9	[7,5,4,6,1,4,1,6]	4	1	Não
10	[7,5,2,4,6,4,8,7]	2	2	Não
11	[8,3,5,3,4,5,7,7]	4	1	Não
12	[6,4,3,8,2,1,2,3]	3	1	Não
13	[3,7,2,7,7,8,5,1]	2	2	Não
14	[2,5,6,2,5,7,3,8]	3	2	Não
15	[1,5,3,2,5,4,3,6]	3	2	Não
16	[3,1,6,8,1,2,6,5]	2	1	Não
17	[4,1,4,2,2,8,2,3]	3	1	Não
18	[3,8,3,5,7,4,4,5]	3	1	Não
19	[7,6,8,8,2,4,4,2]	4	2	Não
20	[6,1,4,4,1,2,1,4]	5	1	Não

- **Soluções encontradas:** 2 de 20.
- O Random Restart melhora a completude prática, porque reinicia a busca em regiões diferentes da superfície.

3.4. Simulated Annealing

Parâmetros utilizados:

```

T0 = 15.0
resfriamento: T <- 0.97T
Tmin = 0.001
semente = 99

```

Resultado:

Estado final: [3, 6, 8, 1, 5, 7, 2, 4]

$h(s)$ final: 0

Passos: 317

Exemplos de movimentos piores aceitos no início da busca:

Passo	h atual	h novo	ΔE	T	Probabilidade aproximada
4	6	7	1	13.279	0.927
5	7	9	2	12.881	0.856
6	9	12	3	12.495	0.787
8	10	11	1	11.756	0.918
10	9	10	1	11.061	0.914

O Simulated Annealing aceita alguns movimentos piores com probabilidade $P = e^{-\Delta E/T}$. No começo, com temperatura alta, essa probabilidade é maior; no final, com temperatura baixa, o algoritmo se comporta mais como Hill-Climbing.

3.5. Comparação dos três algoritmos

Critério	Hill-Climbing	Random Restart HC	Simulated Annealing
Resultado obtido	Parou em [2, 8, 1, 7, 4, 6, 1, 5], com $h(s) = 1$	Encontrou 2 soluções em 20 execuções	Encontrou solução com $h(s) = 0$
Qualidade das soluções	Limitada, pois ficou preso em um mínimo local	Melhor que o Hill-Climbing simples, mas com baixa taxa de sucesso na amostra	Boa, pois encontrou uma solução global na execução realizada
Velocidade de convergência	Rápida, com 6 iterações	Moderada, pois executa várias buscas independentes	Mais lenta, com 317 passos
Sensibilidade ao estado inicial	Alta	Menor, pois utiliza reinícios aleatórios	Menor, pois pode aceitar movimentos piores durante a busca
Capacidade de escapar de mínimos locais	Nenhuma	Via reinícios aleatórios	Via probabilidade de aceitação $P = e^{-\Delta E/T}$
Custo computacional	Baixo	Moderado, pois executa 20 tentativas	Alto, devido ao maior número de passos e ao controle de temperatura
Estabilidade dos resultados	Baixa	Moderada, dependendo dos estados iniciais sorteados	Depende da temperatura inicial, da política de resfriamento e da semente
Conclusão	Simple e rápido, mas pode parar sem solução	Aumenta a chance de encontrar solução, mas não garante sucesso em poucas execuções	Mais robusto na execução realizada, porém mais custoso

4. Questão 04 — CSP e otimizações de busca

4.1. Modelagem como CSP

- a) **Conjunto de variáveis:** T1, T2, T3, T4, T5, T6.
- b) **Domínio de cada variável:** {A, B, C, D}.
- c) **Restrições:**

Nº	Restrição	Tipo
1	$T_i \neq T_{i+1}$, para $i = 1, \dots, 5$	Binária
2	$T_3 \neq A$	Unária
3	$T_1 = B$ ou $T_2 = B$	Binária
4	Não pode ocorrer simultaneamente $T_2 = C$ e $T_5 = C$	Binária
5	O médico D pode aparecer no máximo duas vezes	Global

- f) O grafo de restrições possui arestas entre turnos consecutivos, além de relações extras entre T1–T2, T2–T5 e uma restrição global envolvendo todas as variáveis para contar as ocorrências de D.

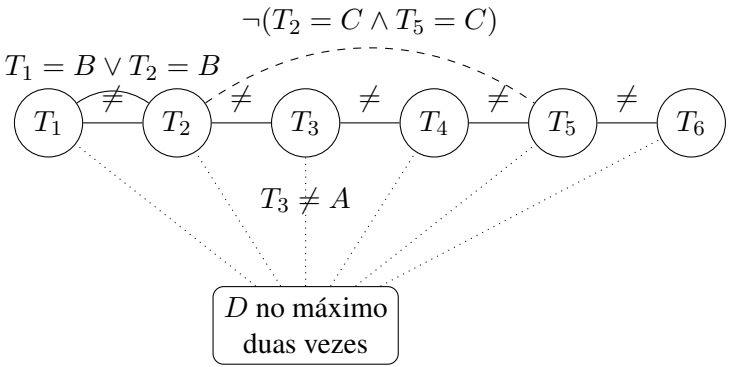


Figura 14. Grafo de restrições do problema de escala de médicos.

4.2. Backtracking manual

Ordem de variáveis: T1, T2, T3, T4, T5, T6. Ordem de valores: A, B, C, D.

- a) **Árvore parcial de busca:**

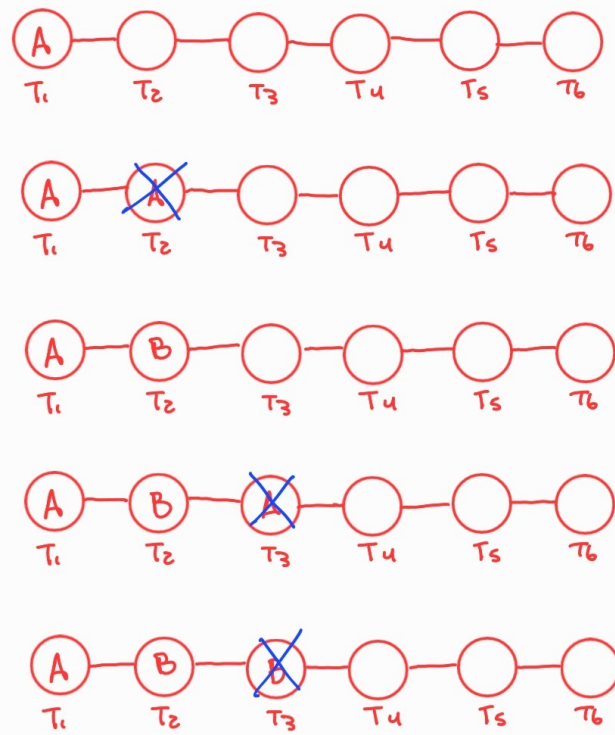


Figura 15. Árvore de busca CSP 1

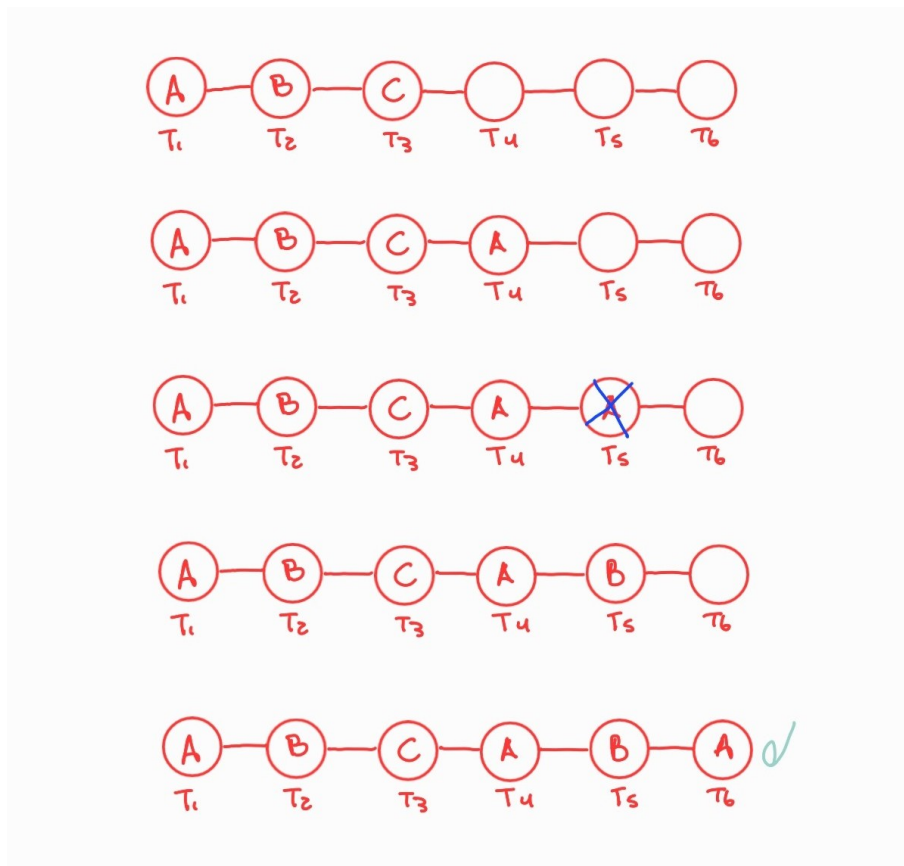


Figura 16. Árvore de busca CSP 2

Passo	Variável atribuída	Estado parcial	Resultado
1	T1=A	{T1=A}	Válido
2	T2=A	{T1=A, T2=A}	Conflito: turnos consecutivos iguais
3	T2=B	{T1=A, T2=B}	Válido
4	T3=A	{T1=A, T2=B, T3=A}	Conflito: A não pode trabalhar em T3
5	T3=B	{T1=A, T2=B, T3=B}	Conflito: T2 e T3 consecutivos iguais
6	T3=C	{T1=A, T2=B, T3=C}	Válido
7	T4=A	{T1=A, T2=B, T3=C, T4=A}	Válido
8	T5=A	{T1=A, T2=B, T3=C, T4=A, T5=A}	Conflito: turnos consecutivos iguais
9	T5=B	{T1=A, T2=B, T3=C, T4=A, T5=B}	Válido
10	T6=A	{T1=A, T2=B, T3=C, T4=A, T5=B, T6=A}	Solução

f) **Solução encontrada:**

$T1=A, T2=B, T3=C, T4=A, T5=B, T6=A$

4.3. MRV e Degree Heuristic

Com MRV, a variável mais restrita inicialmente é T3, porque A é removido do seu domínio. Assim:

$\text{Dom}(T3) = \{B, C, D\}$

$\text{Dom}(\text{demais}) = \{A, B, C, D\}$

O MRV contribui para reduzir o espaço de busca ao priorizar variáveis com menos valores possíveis, enquanto o Degree Heuristic auxilia na escolha de variáveis mais conectadas, capazes de restringir mais rapidamente as próximas atribuições.

Passo	Variável atribuída	Heurística usada	Estado parcial	Resultado
1	$T_3 = B$	MRV: T_3 possui domínio reduzido, pois $T_3 \neq A$	$\{T_3 = B\}$	Válido
2	$T_1 = B$	MRV: após $T_3 = B$, $T_2 \neq B$, então T_1 deve ser B para satisfazer $T_1 = B \vee T_2 = B$	$\{T_3 = B, T_1 = B\}$	Válido
3	$T_2 = A$	Degree Heuristic: entre variáveis empatadas, T_2 participa de mais restrições relevantes	$\{T_3 = B, T_1 = B, T_2 = A\}$	Válido
4	$T_4 = A$	MRV: T_4 não pode receber B , pois $T_3 = B$	$\{T_3 = B, T_1 = B, T_2 = A, T_4 = A\}$	Válido
5	$T_5 = B$	MRV: T_5 não pode ser igual a T_4	$\{T_3 = B, T_1 = B, T_2 = A, T_4 = A, T_5 = B\}$	Válido
6	$T_6 = A$	Última variável restante, respeitando $T_5 \neq T_6$	$\{T_3 = B, T_1 = B, T_2 = A, T_4 = A, T_5 = B, T_6 = A\}$	Solução

A solução obtida com MRV + Degree Heuristic foi:

$T_1=B, T_2=A, T_3=B, T_4=A, T_5=B, T_6=A$

4.4. Forward Checking

O Forward Checking antecipa inconsistências porque remove valores incompatíveis dos domínios futuros. Se algum domínio fica vazio, a busca retrocede imediatamente.

a) Redução de domínios após cada atribuição:

Atribuição	Reduções principais
$T_1=A$	Remove A de Dom(T_2). Como T_1 não é B, força $T_2=B$.
$T_2=B$	Remove B de Dom(T_3).
$T_3=C$	Remove C de Dom(T_4).
$T_4=A$	Remove A de Dom(T_5).
$T_5=B$	Remove B de Dom(T_6).

b) Domínios eliminados:

- T_2 : perdeu $\{A\}$ por $T_1 = A$ consecutivo; depois foi restringido a $\{B\}$, pois $T_1 \neq B$ e a restrição $T_1 = B \vee T_2 = B$ deve ser satisfeita.
- T_3 : perdeu $\{B\}$ por $T_2 = B$ consecutivo.
- T_4 : perdeu $\{C\}$ por $T_3 = C$ consecutivo.
- T_5 : perdeu $\{A\}$ por $T_4 = A$ consecutivo.
- T_6 : perdeu $\{B\}$ por $T_5 = B$ consecutivo.

- c) **Inconsistências detectadas:** As inconsistências foram detectadas antecipadamente, antes de testar valores inválidos. Por exemplo, após $T_1 = A$, o algoritmo já elimina $T_2 = A$ e força $T_2 = B$. Da mesma forma, após cada atribuição, o valor igual ao turno anterior é removido do domínio do próximo turno.
- d) **Comparação com backtracking simples:** O Forward Checking é mais eficiente que o backtracking simples, pois evita testar valores que já se sabe que causarão conflito. Assim, reduz o espaço de busca e diminui a necessidade de retrocessos.

4.5. Backjumping

O Backjumping evita retrocessos desnecessários, principalmente quando o conflito depende de uma variável distante na ordem cronológica.

a) Implementação do Backjumping:

Passo	Tentativa	Estado parcial	Conflito / verificação	Variável causadora	Ação do Backjumping
1	$T_1 = A$	$\{T_1 = A\}$	Atribuição válida	–	Avança para T_2
2	$T_2 = A$	$\{T_1 = A, T_2 = A\}$	Conflito: $T_1 = T_2$ e $T_1 = B \vee T_2 = B$ não é satisfeita	T_1	Rejeita A em T_2 e testa próximo valor
3	$T_2 = B$	$\{T_1 = A, T_2 = B\}$	Atribuição válida; satisfaz $T_1 = B \vee T_2 = B$	–	Avança para T_3
4	$T_3 = A$	$\{T_1 = A, T_2 = B, T_3 = A\}$	Conflito: A não pode trabalhar em T_3	Restrição unária de T_3	Rejeita A em T_3 e testa próximo valor
5	$T_3 = B$	$\{T_1 = A, T_2 = B, T_3 = B\}$	Conflito: turnos consecutivos iguais, pois $T_2 = T_3$	T_2	Rejeita B em T_3 e testa próximo valor
6	$T_3 = C$	$\{T_1 = A, T_2 = B, T_3 = C\}$	Atribuição válida	–	Avança para T_4
7	$T_4 = A$	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A\}$	Atribuição válida, pois $T_3 \neq T_4$	–	Avança para T_5
8	$T_5 = A$	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = A\}$	Conflito: turnos consecutivos iguais, pois $T_4 = T_5$	T_4	Rejeita A em T_5 e testa próximo valor
9	$T_5 = B$	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = B\}$	Atribuição válida; também não viola $\neg(T_2 = C \wedge T_5 = C)$	–	Avança para T_6
10	$T_6 = A$	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = B, T_6 = A\}$	Atribuição válida, pois $T_5 \neq T_6$	–	Solução encontrada

- c) **Diferenças para backtracking tradicional:** No backtracking tradicional, ao encontrar uma inconsistência, o algoritmo volta apenas para a variável imediatamente anterior. No Backjumping, volta diretamente para uma variável responsável pelo conflito.
- d) **Redução observada na árvore de busca:** Com base na tabela, observa-se que o Backjumping reduziu pouco a árvore de busca, pois os conflitos encontrados foram locais e envolveram principalmente variáveis consecutivas. Assim, quando ocorreu conflito em $T_2 = A$, $T_3 = B$ ou $T_5 = A$, o algoritmo apenas rejeitou o valor testado e continuou avaliando o próximo valor da mesma variável.

Neste caminho de execução, não houve salto longo, pois nenhuma variável teve seu domínio totalmente esgotado antes de a solução ser encontrada. Por isso, o comportamento do Backjumping foi semelhante ao do backtracking simples. A principal vantagem potencial do Backjumping apareceria caso fosse violada a restrição $\neg(T_2 = C \wedge T_5 = C)$. Nesse caso, o algoritmo poderia saltar diretamente de T_5 para T_2 , ignorando variáveis intermediárias que não causaram o conflito.

4.6. Comparação dos algoritmos:

Critério	Backtracking simples	Backtracking + MRV + Degree	Forward Checking	Backjumping
Estados explorados	Alto; na execução, foram feitas 10 tentativas de atribuição.	Menor que o simples, pois escolhe variáveis mais restritas e, em caso de empate, mais conectadas.	Reduzido, pois evita testar valores já incompatíveis.	Semelhante ao simples nesta execução, pois os conflitos foram locais.
Retrocessos	Maior; ocorreram conflitos em $T_2 = A$, $T_3 = A$, $T_3 = B$ e $T_5 = A$.	Menor, pois tende a detectar falhas mais cedo.	Baixo, pois remove valores inválidos antes de novas atribuições.	Pode ser menor quando há conflitos com variáveis distantes.
Velocidade de execução	Baixa a moderada, pois testa valores inválidos antes de descartá-los.	Boa, pois combina falha precoce com maior propagação de restrições.	Boa, pois antecipa inconsistências nos domínios futuros.	Boa em conflitos não locais; nesta execução, o ganho foi pequeno.
Consumo de memória	Baixo, pois armazena basicamente a pilha de chamadas e o estado parcial.	Moderado, pois considera domínios restantes e grau das variáveis.	Moderado, pois mantém e atualiza domínios das variáveis não atribuídas.	Baixo a moderado, pois precisa armazenar conjuntos de conflito.
Facilidade de implementação	Alta; é o mais simples de implementar.	Moderada, pois exige controle dos domínios e cálculo do grau das variáveis.	Moderada, pois exige atualização de domínios após cada atribuição.	Menor, pois exige identificar variáveis causadoras dos conflitos.
Eficiência prática	Baixa a moderada, pois pode explorar muitos ramos desnecessários.	Boa, especialmente em problemas com muitas restrições.	Muito boa, pois reduz significativamente o espaço de busca.	Boa, principalmente quando há restrições entre variáveis distantes, como T_2 e T_5 .

5. Questão 05 — Minimax e poda Alpha-Beta

5.1. Conceitos

- Objetivo do algoritmo Minimax:** escolher a melhor jogada em jogos de dois jogadores, considerando que o adversário também jogará da melhor forma possível. Ele simula as possíveis jogadas futuras e escolhe a ação que maximiza o ganho do jogador atual, assumindo que o oponente tentará minimizar esse ganho. Portanto, busca a melhor decisão possível em um cenário competitivo.
- Diferença entre nós MAX e MIN:** nó MAX representa a vez do jogador que queremos favorecer. Ele escolhe o maior valor entre seus filhos, pois busca maximizar sua chance de vitória ou sua pontuação. Já o nó MIN representa a vez

do adversário. Ele escolhe o menor valor entre seus filhos, pois assume-se que o adversário tentará minimizar o resultado do jogador MAX.

- c) **Utilidade:** é o valor numérico que representa o quão bom ou ruim é um estado do jogo para o jogador MAX. Ela normalmente é atribuída aos estados finais da árvore.
- d) **Propagação de valores na árvore:** propaga valores de baixo para cima, usando máximo nos nós MAX e mínimo nos nós MIN.
- e) **Hipótese de adversário perfeito:** MIN sempre escolherá o pior resultado possível para MAX.
- f) **Objetivo da poda Alpha-Beta:** tornar o algoritmo Minimax mais eficiente, evitando explorar ramos da árvore que não podem alterar a decisão final. Ela elimina jogadas que já se sabe que serão descartadas, porque existe uma alternativa melhor encontrada anteriormente.

5.2. Execução do Minimax

Nó	Tipo	Ordem de expansão	Ordem de cálculo	Cálculo	Valor Minimax
A	MAX	1	10	$\max(B, C, D) = \max(5, 0, 6)$	6
B	MIN	2	3	$\min(E, F) = \min(5, 9)$	5
E	MAX	3	1	$\max(3, 5)$	5
F	MAX	4	2	$\max(6, 9)$	9
C	MIN	5	6	$\min(G, H) = \min(2, 0)$	0
G	MAX	6	4	$\max(1, 2)$	2
H	MAX	7	5	$\max(0, -1)$	0
D	MIN	8	9	$\min(I, J) = \min(7, 6)$	6
I	MAX	9	7	$\max(7, 4)$	7
J	MAX	10	8	$\max(5, 6)$	6

O valor Minimax da raiz é 6. Portanto, a melhor jogada para o jogador MAX é escolher o terceiro ramo da raiz, correspondente ao nó *D*. Considerando a resposta ótima do jogador MIN, o caminho escolhido é:

$$A \rightarrow D \rightarrow J \rightarrow 6.$$

- f) **Árvore parcial de busca:**

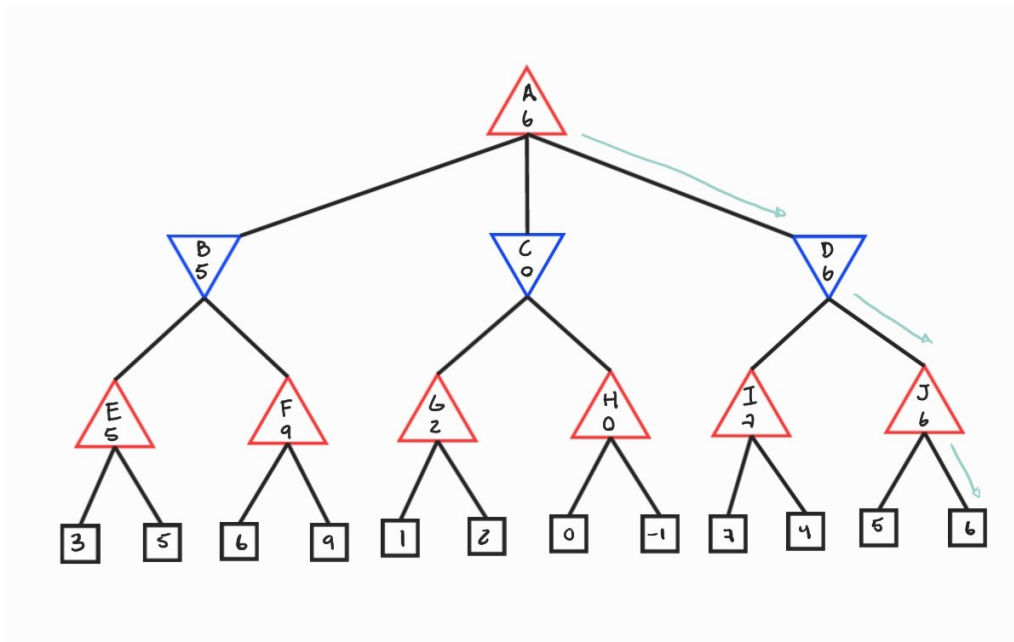


Figura 17. Árvore de busca Minimax

5.3. Execução do Minimax com poda Alpha-Beta na ordem original

Nó	Tipo	Ordem de expansão	Ordem de cálculo	α	β	Cálculo realizado	Valor	Poda
A	MAX	1	9	$-\infty \rightarrow 6$	$+\infty$	$\max(B, C, D) = \max(5, 2, 6)$	6	Não ocorre poda na raiz
B	MIN	2	3	$-\infty$	$+\infty \rightarrow 5$	$\min(E, F) = \min(5, 6)$	5	Não ocorre poda em B
E	MAX	3	1	$-\infty \rightarrow 5$	$+\infty$	$\max(3, 5)$	5	Não ocorre poda
F	MAX	4	2	$-\infty \rightarrow 6$	5	Avalia 6. Como $6 \geq \beta = 5$, não precisa avaliar o próximo filho	6	Poda a folha 9
C	MIN	5	5	5	$+\infty \rightarrow 2$	Avalia $G = 2$. Como $2 \leq \alpha = 5$, não precisa avaliar H	2	Poda o ramo H, isto é, as folhas 0 e -1
G	MAX	6	4	5	$+\infty$	$\max(1, 2)$	2	Não ocorre poda
D	MIN	7	8	5	$+\infty \rightarrow 6$	$\min(I, J) = \min(7, 6)$	6	Não ocorre poda
I	MAX	8	6	$5 \rightarrow 7$	$+\infty$	$\max(7, 4)$	7	Não ocorre poda
J	MAX	9	7	$5 \rightarrow 6$	7	$\max(5, 6)$	6	Não ocorre poda

- e) **Quantidade de nós não explorados:** 4 nós: a folha 9, o nó H e suas duas folhas (0, -1).

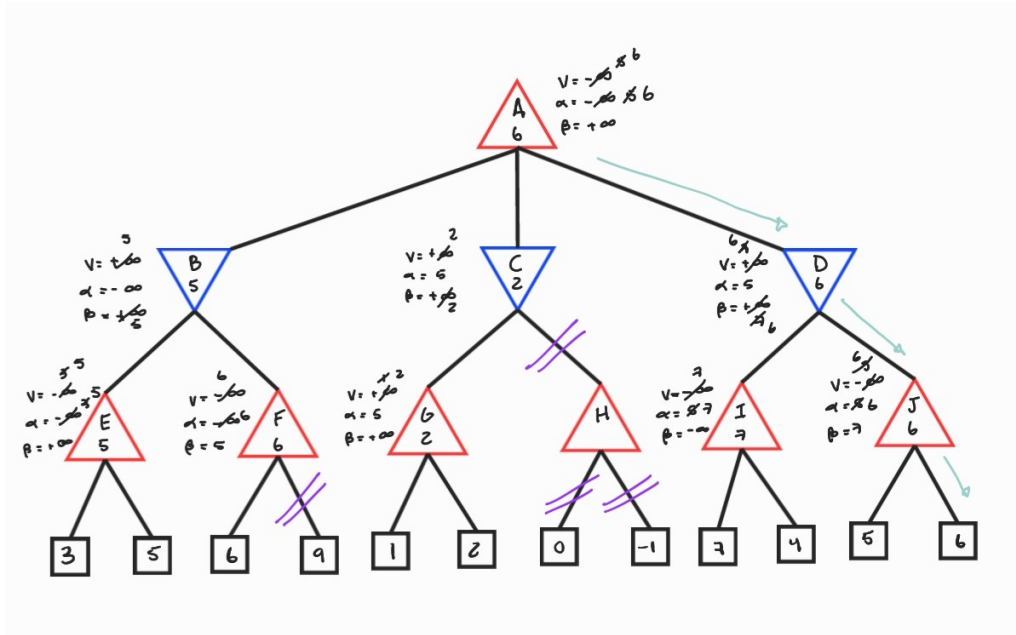


Figura 18. Árvore de busca Minimax com poda Alpha-Beta na ordem original

5.4. Reordenação para maximizar podas

Para maximizar o número de podas na execução do algoritmo Alpha-Beta, é conveniente reordenar os nós de modo que os melhores ramos sejam avaliados primeiro. Assim, nos nós do tipo MAX, são priorizados os filhos com maiores valores esperados, enquanto nos nós do tipo MIN são priorizados os filhos com menores valores esperados. Com essa ordenação, os limites α e β são atualizados mais cedo, aumentando a possibilidade de eliminar ramos que não influenciam a decisão final.

A ordem utilizada para maximizar as podas foi:

$$A : D, B, C$$

$$D : J, I \quad B : E, F \quad C : H, G$$

Execução do algoritmo com os nós reordenados:

Nó	Tipo	Ordem de expansão	Ordem de cálculo	α	β	Cálculo realizado	realizado	Valor	Poda
A	MAX	1	8	$-\infty \rightarrow 6$	$+\infty$	$\max(D, B, C) = 6$ $\max(6, 5, 0)$			Não ocorre poda na raiz

Nó	Tipo	Ordem de expansão	Ordem de cálculo	α	β	Cálculo realizado	realizado	Valor	Poda
D	MIN	2	3	$-\infty$	$+\infty \rightarrow 6$	$\min(J, I)$ $\min(6, 7)$	=	6	Não poda filhos diretos
J	MAX	3	1	$-\infty \rightarrow 6$	$+\infty$	$\max(6, 5)$		6	Não ocorre poda
I	MAX	4	2	$-\infty \rightarrow 7$	6	Avalia 7. Como $7 \geq \beta = 6$, o próximo filho não altera a decisão		7	Poda a folha 4
B	MIN	5	5	6	$+\infty \rightarrow 5$	Avalia $E = 5$. Como $5 \leq \alpha = 6$, o outro filho não precisa ser explorado		5	Poda o ramo F , isto é, as folhas 9 e 6
E	MAX	6	4	6	$+\infty$	$\max(5, 3)$		5	Não ocorre poda
C	MIN	7	7	6	$+\infty \rightarrow 0$	Avalia $H = 0$. Como $0 \leq \alpha = 6$, o outro filho não precisa ser explorado		0	Poda o ramo G , isto é, as folhas 2 e 1
H	MAX	8	6	6	$+\infty$	$\max(0, -1)$		0	Não ocorre poda

- Eventos de poda antes:** 2.
- Eventos de poda depois:** 3.
- Por que a nova ordem melhora a eficiência?:** A nova ordem melhora a eficiência porque Alpha-Beta depende fortemente da ordem de expansão. Quanto mais cedo bons valores para MAX e baixos valores para MIN são encontrados, mais cedo as condições $\alpha \geq \beta$ ou $\beta \leq \alpha$ aparecem.
- Qual a relação entre ordenação e desempenho?:** Uma boa ordenação reduz o número de nós avaliados e torna o algoritmo mais eficiente. Já uma má ordenação diminui o número de podas e faz o desempenho se aproximar do Minimax tradicional.

5.5. Minimax com profundidade limitada

Nó	Tipo	Prof.	Ordem de expansão	Ordem de cálculo	Cálculo / avaliação	Valor
A	MAX	0	1	10	$\max(B, C, D) = \max(4, 2, 1)$	4
B	MIN	1	2	3	$\min(E, F) = \min(4, 7)$	4
E	MAX	2	3	1	Nó de corte: $h(E) = 4$	4
F	MAX	2	4	2	Nó de corte: $h(F) = 7$	7

Nó	Tipo	Prof.	Ordem de expansão	Ordem de cálculo	Cálculo / avaliação	Valor
C	MIN	1	5	6	$\min(G, H) = \min(2, 6)$	2
G	MAX	2	6	4	Nó de corte: $h(G) = 2$	2
H	MAX	2	7	5	Nó de corte: $h(H) = 5$	5
D	MIN	1	8	9	$\min(I, J) = \min(6, 1)$	1
I	MAX	2	9	7	Nó de corte: $h(I) = 6$	6
J	MAX	2	10	8	Nó de corte: $h(J) = 1$	1

- c) **Comparação com minimax completo:** o Minimax completo escolhe o terceiro ramo com valor 6, enquanto o Minimax limitado escolhe o primeiro ramo com valor heurístico 4.
- d) **Erro causado pela heurística:** a heurística subestimou ou distorceu os valores dos ramos, fazendo o algoritmo escolher uma jogada diferente da ótima. Isso mostra que, em buscas limitadas, a qualidade da função de avaliação é decisiva.

5.6. Comparação entre Minimax e Poda Alpha-Beta

Critério	Minimax completo	Alpha-Beta
Número de nós explorados	Explora todos os nós da árvore: 10 nós internos e 12 folhas, totalizando 22 nós.	Explora menos nós: 9 nós internos e 9 folhas, totalizando 18 nós.
Quantidade de podas	Não realiza podas.	Realiza podas em 3 folhas. Se contar também o nó interno H , são 4 nós não explorados.
Custo computacional	Maior, pois avalia toda a árvore de jogo.	Menor que o Minimax, pois evita avaliar ramos que não alteram a decisão final.
Consumo de memória	Depende da profundidade da recursão; em busca em profundidade, é proporcional à profundidade da árvore.	Semelhante ao Minimax em termos de pilha recursiva, mas pode economizar processamento por explorar menos nós.
Impacto da ordenação dos movimentos	A ordenação não altera a quantidade de nós avaliados, pois todos os ramos são explorados.	A ordenação influencia diretamente o desempenho: melhor ordem gera mais podas.
Qualidade das decisões	Encontra a decisão ótima, assumindo adversário perfeito.	Encontra a mesma decisão ótima do Minimax, pois a poda não altera o resultado final.

5.7. Experimento adicional

Neste experimento, foram modificados os valores de exatamente três folhas da árvore. Os valores terminais originais eram:

[3, 5, 6, 9, 1, 2, 0, -1, 7, 4, 5, 6]

Foram realizadas as seguintes alterações:

$$3 \rightarrow 8, \quad 5 \rightarrow 8, \quad -1 \rightarrow 4.$$

Assim, os novos valores terminais passaram a ser:

$$[8, 8, 6, 9, 1, 2, 0, 4, 7, 4, 5, 6]$$

Minimax após a modificação de três folhas

Nó	Tipo	Ordem de expansão	Ordem de cálculo	Cálculo	Valor
A	MAX	1	10	$\max(B, C, D) = \max(8, 2, 6)$	8
B	MIN	2	3	$\min(E, F) = \min(8, 9)$	8
E	MAX	3	1	$\max(8, 8)$	8
F	MAX	4	2	$\max(6, 9)$	9
C	MIN	5	6	$\min(G, H) = \min(2, 4)$	2
G	MAX	6	4	$\max(1, 2)$	2
H	MAX	7	5	$\max(0, 4)$	4
D	MIN	8	9	$\min(I, J) = \min(7, 6)$	6
I	MAX	9	7	$\max(7, 4)$	7
J	MAX	10	8	$\max(5, 6)$	6

Com os novos valores, o valor Minimax da raiz passa a ser 8. Portanto, a melhor jogada para MAX deixa de ser o ramo D e passa a ser o ramo B .

Alpha-Beta após a modificação de três folhas

Nó	Tipo	Ordem de expansão	Ordem de cálculo	α	β	Cálculo realizado	Valor	Poda
A	MAX	1	8	$-\infty \rightarrow 8$	$+\infty$	$B = 8, C \leq 2, D \leq 7$. Logo, $A = 8$	8	Não ocorre poda na raiz
B	MIN	2	3	$-\infty$	$+\infty \rightarrow 8$	$\min(E, F) = \min(8, 9)$	8	Não ocorre poda efetiva
E	MAX	3	1	$-\infty \rightarrow 8$	$+\infty$	$\max(8, 8)$	8	Não ocorre poda
F	MAX	4	2	$-\infty \rightarrow 9$	8	$\max(6, 9)$	9	Não há filho restante a podar

Nó	Tipo	Ordem de expansão	Ordem de cálculo	α	β	Cálculo realizado	realizado	Valor	Poda
C	MIN	5	5	8	$+\infty \rightarrow 2$	Após $G = 2$, tem-se $2 \leq \alpha = 8$	≤ 2		Poda o ramo H , isto é, as folhas 0 e 4
G	MAX	6	4	8	$+\infty$	$\max(1, 2)$		2	Não ocorre poda
D	MIN	7	7	8	$+\infty \rightarrow 7$	Após $I = 7$, tem-se $7 \leq \alpha = 8$	≤ 7		Poda o ramo J , isto é, as folhas 5 e 6
I	MAX	8	6	8	$+\infty$	$\max(7, 4)$		7	Não ocorre poda

Na execução com Alpha-Beta, a decisão final permanece a mesma do Minimax, isto é, o ramo escolhido é B , com valor 8. Entretanto, o Alpha-Beta evita avaliar os ramos H e J , pois eles não poderiam alterar a decisão final da raiz.

Verificação da decisão final

Antes da modificação, os valores principais eram:

$$B = 5, \quad C = 0, \quad D = 6.$$

Logo, MAX escolhia o ramo D , pois:

$$A = \max(5, 0, 6) = 6.$$

Após a modificação, os valores passaram a ser:

$$B = 8, \quad C = 2, \quad D = 6.$$

Assim:

$$A = \max(8, 2, 6) = 8.$$

Portanto, a decisão final mudou: antes MAX escolhia D , e depois passou a escolher B .

Sensibilidade às utilidades

O experimento mostra que o Minimax é sensível aos valores das folhas, pois alterações nas utilidades terminais podem modificar a propagação dos valores na árvore e, conseqüentemente, a decisão final. Neste caso, ao aumentar os valores das folhas associadas

ao ramo B , seu valor Minimax passou de 5 para 8, superando o valor do ramo D . A poda Alpha-Beta manteve a mesma decisão do Minimax, mas com menor número de nós avaliados.

6. Questão 06 – Monte Carlo Tree Search (MCTS)

Considera-se o jogo Connect-4 simplificado em um tabuleiro 4×4 . O jogador vermelho, representado por V , deve escolher a próxima coluna para jogar. O estado inicial considerado é:

A	V		
V	A		

As ações possíveis são:

$$c_1, c_2, c_3, c_4.$$

Nesta resolução, a recompensa usada na retropropagação foi:

$$\text{vitória} = 1, \quad \text{empate} = 0,5, \quad \text{derrota} = 0.$$

6.1. Conceitos

- Seleção:** a busca começa na raiz, que representa o estado atual do jogo. O algoritmo escolhe sucessivamente os filhos mais promissores, geralmente usando a fórmula UCT, que equilibra exploração de nós pouco visitados e exploração de nós com boa taxa de vitória.
- Expansão:** ao chegar a um nó com ações ainda não testadas, o algoritmo cria um novo filho correspondente a uma dessas ações. Assim, a árvore cresce gradualmente nas regiões mais promissoras.
- Simulação ou rollout:** a partir do nó expandido, uma partida é simulada até o final ou até uma condição de parada. Nesta primeira execução, o rollout utilizado foi aleatório.
- Retropropagação:** o resultado da simulação é propagado de volta pelos nós visitados no caminho. Em cada nó, são atualizados o número de visitas $N(s)$ e a recompensa acumulada $W(s)$.
- Papel do número de visitas:** indica quantas vezes um nó já foi explorado. Nós pouco visitados ainda possuem estimativas incertas, enquanto nós muito visitados têm avaliações mais confiáveis.
- Papel do número de vitórias:** indica quantas simulações que passaram por aquele nó terminaram favoravelmente. Esse valor ajuda a estimar a qualidade média da jogada.
- Diferença entre exploração e exploração:** exploração significa testar jogadas pouco avaliadas; exploração significa priorizar jogadas que já apresentaram bons resultados. O MCTS busca equilibrar esses dois comportamentos.

6.2. Execução de 10 iterações com rollout aleatório

Foi utilizado $C = 1,4$. Como o rollout é aleatório, os resultados podem variar entre execuções. A tabela a seguir apresenta uma execução possível e reprodutível para fins de análise.

Iter.	Caminho selecionado	Nó expandido	Resultado	Atualização da raiz	Filhos da raiz após a iteração
1	$R \rightarrow c_1$	c_1	0	$N(R) = 1, W(R) = 0$	$c_1 : (N = 1, W = 0)$
2	$R \rightarrow c_2$	c_2	1	$N(R) = 2, W(R) = 1$	$c_1 : (1, 0), c_2 : (1, 1)$
3	$R \rightarrow c_3$	c_3	0	$N(R) = 3, W(R) = 1$	$c_1 : (1, 0), c_2 : (1, 1), c_3 : (1, 0)$
4	$R \rightarrow c_4$	c_4	0	$N(R) = 4, W(R) = 1$	$c_1 : (1, 0), c_2 : (1, 1), c_3 : (1, 0), c_4 : (1, 0)$
5	$R \rightarrow c_2 \rightarrow c_1$	$c_2 c_1$	1	$N(R) = 5, W(R) = 2$	$c_1 : (1, 0), c_2 : (2, 2), c_3 : (1, 0), c_4 : (1, 0)$
6	$R \rightarrow c_2 \rightarrow c_2$	$c_2 c_2$	0	$N(R) = 6, W(R) = 2$	$c_1 : (1, 0), c_2 : (3, 2), c_3 : (1, 0), c_4 : (1, 0)$
7	$R \rightarrow c_1 \rightarrow c_1$	$c_1 c_1$	1	$N(R) = 7, W(R) = 3$	$c_1 : (2, 1), c_2 : (3, 2), c_3 : (1, 0), c_4 : (1, 0)$
8	$R \rightarrow c_3 \rightarrow c_1$	$c_3 c_1$	0,5	$N(R) = 8, W(R) = 3,5$	$c_1 : (2, 1), c_2 : (3, 2), c_3 : (2, 0,5), c_4 : (1, 0)$
9	$R \rightarrow c_4 \rightarrow c_1$	$c_4 c_1$	0,5	$N(R) = 9, W(R) = 4$	$c_1 : (2, 1), c_2 : (3, 2), c_3 : (2, 0,5), c_4 : (2, 0,5)$
10	$R \rightarrow c_1 \rightarrow c_2$	$c_1 c_2$	0,5	$N(R) = 10, W(R) = 4,5$	$c_1 : (3, 1,5), c_2 : (3, 2), c_3 : (2, 0,5), c_4 : (2, 0,5)$

Após as 10 iterações, os filhos da raiz ficaram com os seguintes valores:

$$c_1 : (N = 3, W = 1,5), \quad c_2 : (N = 3, W = 2), \quad c_3 : (N = 2, W = 0,5), \quad c_4 : (N = 2, W = 0,5).$$

6.3. Cálculo do UCT com os valores obtidos na execução

A fórmula utilizada é:

$$UCT(j) = \frac{W_j}{N_j} + C \sqrt{\frac{\ln N}{N_j}},$$

em que W_j é a recompensa acumulada do filho, N_j é o número de visitas do filho e N é o número de visitas da raiz.

Após 10 iterações:

$$N = 10.$$

Com $C = 1,4$, os valores de UCT dos filhos da raiz são:

a) **Valores de UCT após as 10 iterações:**

Jogada	N_j	W_j	W_j/N_j	Exploração	UCT	Interpretação
c_1	3	1,5	0,50	1,227	1,727	Recompensa média intermediária
c_2	3	2	0,67	1,227	1,893	Maior UCT e melhor média de recompensa
c_3	2	0,5	0,25	1,502	1,752	Menor recompensa, mas favorecido pela exploração
c_4	2	0,5	0,25	1,502	1,752	Menor recompensa, mas favorecido pela exploração

Portanto, usando os valores obtidos na própria execução do algoritmo, o maior UCT após a décima iteração é o de c_2 :

$$UCT(c_2) = 1,893.$$

Assim, se uma nova iteração fosse realizada, o algoritmo selecionaria inicialmente o ramo c_2 .

b) **Nó selecionado:** O nó selecionado pelo MCTs é aquele com maior número de visitas. O c_1 e o c_2 ficaram empatados com 3 visitas. Sendo assim, ambos são candidatos à melhor jogada.

c) **Influência da exploração e da exploração:**

O termo

$$\frac{W_j}{N_j}$$

representa a exploração, pois favorece jogadas com maior recompensa média.

O termo

$$C \sqrt{\frac{\ln N}{N_j}}$$

representa a exploração, pois favorece jogadas pouco visitadas. Quanto menor N_j , maior é esse termo.

6.4. Comparação entre diferentes valores de C

A constante C controla o equilíbrio entre exploração e exploração. Para comparar seu efeito, foram consideradas execuções com $C = 0,1$, $C = 1,4$ e $C = 3,0$.

Valor de C	Comportamento	Resultado após 10 iterações
$C = 0,1$	Predomínio da exploração.	$c_1 : (1, 0)$, $c_2 : (7, 5)$, $c_3 : (1, 0)$, $c_4 : (1, 0)$. Jogada escolhida: c_2 .
$C = 1,4$	Equilíbrio entre exploração e exploração.	$c_1 : (3, 1, 5)$, $c_2 : (3, 2)$, $c_3 : (2, 0, 5)$, $c_4 : (2, 0, 5)$. Jogada escolhida: c_2 .

Valor de C	Comportamento	Resultado após 10 iterações
$C = 3,0$	Predomínio da exploração.	$c_1 : (3, 1), c_2 : (3, 2), c_3 : (2, 0, 5), c_4 : (2, 0, 5)$. Jogada escolhida: c_2 .

Valores pequenos de C fazem o algoritmo explorar pouco e se concentrar rapidamente na melhor jogada observada. Valores altos de C aumentam a diversidade das jogadas testadas, mas podem tornar a decisão menos estável quando o número de iterações é pequeno.

Comparação entre diferentes valores de C

Critério	$C = 0,1$	$C = 1,4$	$C = 3,0$
Quantidade de exploração	Baixa. O algoritmo privilegia rapidamente as jogadas com melhor recompensa média.	Moderada. Há equilíbrio entre testar jogadas novas e aprofundar as mais promissoras.	Alta. O algoritmo força maior exploração de jogadas pouco visitadas.
Diversidade de jogadas	Baixa. As visitas se concentram principalmente em uma jogada.	Boa. Todas as jogadas recebem visitas, mas as melhores ainda são priorizadas.	Alta. As visitas tendem a ficar mais distribuídas entre as opções disponíveis.
Estabilidade das decisões	Pode ser alta se a primeira boa jogada for realmente forte, mas há risco de convergência prematura.	Boa. A decisão tende a ser mais confiável, pois combina exploração e exploração.	Menor em poucas iterações, pois o algoritmo continua explorando bastante.
Qualidade das jogadas encontradas	Pode ser boa, mas depende muito dos primeiros rollouts. Pode ignorar alternativas melhores.	Geralmente melhor, pois evita tanto a exploração excessiva quanto a convergência muito rápida.	Pode melhorar com muitas iterações, mas em poucas simulações pode demorar a consolidar a melhor jogada.
Comportamento observado após 10 iterações	$c_1 : (1, 0), c_2 : (7, 5), c_3 : (1, 0), c_4 : (1, 0)$	$c_1 : (3, 1, 5), c_2 : (3, 2), c_3 : (2, 0, 5), c_4 : (2, 0, 5)$	$c_1 : (3, 1), c_2 : (3, 2), c_3 : (2, 0, 5), c_4 : (2, 0, 5)$
Jogada escolhida	c_2	c_2	c_2

6.5. Comparação entre rollout aleatório e rollout semi-guloso

Foram considerados dois tipos de rollout:

- 1) **Rollout aleatório:** escolhe jogadas aleatórias até o fim da simulação.
- 2) **Rollout semi-guloso:** prioriza jogadas centrais, vitórias imediatas e bloqueios de vitórias imediatas do adversário.

A tabela a seguir compara os resultados obtidos com cada política após 10 iterações, mantendo $C = 1,4$.

Política de rollout	Resultado após 10 iterações	Jogada escolhida	Interpretação
Rollout aleatório	$c_1 : (3, 1, 5), c_2 : (3, 2), c_3 : c_2$ $(2, 0, 5), c_4 : (2, 0, 5)$		Apresenta maior variabilidade, pois as simulações podem seguir jogadas ruins ou pouco realistas.
Rollout semi-guloso	$c_1 : (3, 2), c_2 : (3, 2, 5), c_3 : c_2$ $(2, 1), c_4 : (2, 1)$		Produz estimativas mais consistentes, pois prioriza jogadas taticamente melhores durante as simulações.

O rollout semi-guloso apresenta valores mais altos porque suas simulações evitam algumas jogadas ruins e consideram decisões táticas simples, como ocupar colunas centrais e bloquear ameaças imediatas. Já o rollout aleatório possui maior diversidade, mas precisa de mais simulações para alcançar estimativas confiáveis.

Comparação qualitativa dos rollouts

Critério	Rollout aleatório	Rollout semi-guloso
Qualidade das decisões	Menor, pois muitas simulações podem seguir jogadas ruins.	Maior, pois considera ações taticamente melhores.
Velocidade de convergência	Mais lenta, pois os resultados têm maior variabilidade.	Mais rápida, pois os resultados são mais consistentes.
Número de simulações necessárias	Maior, pois precisa compensar a aleatoriedade.	Menor, pois cada simulação tende a ser mais informativa.
Diversidade das jogadas	Alta, pois explora muitas possibilidades.	Moderada, pois tende a repetir jogadas consideradas boas.
Estabilidade da decisão	Menor, principalmente com poucas iterações.	Maior, pois segue critérios mais consistentes.